УНИВЕРСИТЕТ ИТМО

Домашнее задание № 4
Отказоустойчивость средствами Apache ZooKeeper

Ансамбль из трёх серверов и координатор расчёта пар навыков из ДЗ 3, вариант 9

Выполнил: Вяхирев Иван · ИСУ 465468 · группа J3310

Код, конфигурация, журналы координаторов и выводы команд лежат в репозитории EngData_2026_3, ветка hw4. Все запуски на одном компьютере с Windows и Docker Desktop. RustFS с бакетом hh-lake остался с ДЗ 2 и ДЗ 3.

1. Ансамбль ZooKeeper

Три сервера подняты в Docker из образа zookeeper:3.9.6 файлом `compose.yaml`. Конфигурация `zoo.cfg` одна на все три сервера: строки участников ансамбля у всех одинаковые. Каждый сервер работает в своём контейнере, поэтому у каждого своя папка dataDir, а клиентский порт наружу у них разный: 2181, 2182, 2183. Номер сервера лежит в файле `/data/myid`, его пишет переменная ZOO_MY_ID.

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data
clientPort=2181
4lw.commands.whitelist=srvr,stat,ruok
server.1=zoo1:2888:3888
server.2=zoo2:2888:3888
server.3=zoo3:2888:3888
```

```
docker compose up -d
```

```
NAME      IMAGE              COMMAND                  SERVICE   CREATED              STATUS              PORTS
zoo1      zookeeper:3.9.6    "/docker-entrypoint.…"   zoo1      About a minute ago   Up About a minute   2888/tcp, 3888/tcp, 0.0.0.0:2181->2181/tcp, 8080/tcp
zoo2      zookeeper:3.9.6    "/docker-entrypoint.…"   zoo2      About a minute ago   Up About a minute   2888/tcp, 3888/tcp, 8080/tcp, 0.0.0.0:2182->2181/tcp
zoo3      zookeeper:3.9.6    "/docker-entrypoint.…"   zoo3      About a minute ago   Up 16 seconds       2888/tcp, 3888/tcp, 8080/tcp, 0.0.0.0:2183->2181/tcp
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data
clientPort=2181
4lw.commands.whitelist=srvr,stat,ruok
server.1=zoo1:2888:3888
server.2=zoo2:2888:3888
server.3=zoo3:2888:3888
== zoo1, myid: 1

Mode: leader
== zoo2, myid: 2
Mode: follower
== zoo3, myid: 3
Mode: follower
```

Три сервера запущены, zoo1 лидер, zoo2 и zoo3 последователи, myid 1, 2, 3

Отказ одного сервера

Остановлен лидер zoo1. Оставшиеся два выбрали нового лидера, zoo3, и ансамбль отвечает на запрос `ls /`: два из трёх это большинство.

```
docker compose stop zoo1
```

```
== zoo2, myid: 2
Mode: follower
== zoo3, myid: 3
Mode: leader
[zookeeper]
```

```
[+] Stopping 1/1
 ✔ Container zoo1  Stopped                                                
```

Отказ двух серверов

Остановлен ещё zoo3. Оставшийся zoo2 один из трёх, большинства нет, и он перестаёт обслуживать клиентов: статус не получить, запрос завершается ConnectionLoss.

```
docker compose stop zoo3
```

```
== zoo2, myid: 2
Error contacting service. It is probably not running.
KeeperErrorCode = ConnectionLoss for /
```



Без двух серверов ансамбль не отвечает

```
docker compose start
```

```
== zoo1, myid: 1
Mode: follower
== zoo2, myid: 2
Mode: follower
== zoo3, myid: 3
Mode: leader
```



После запуска оба сервера вернулись, снова один лидер и два последователя

Эфемерный узел

Во втором окне запущен клиент zkCli в отдельном контейнере, в нём создан эфемерный узел `/demo`. С другого сервера узел виден. Затем клиент завершён командой `docker rm -f zkcli`. Пинги прекратились, через таймаут сессии клиента, 30 секунд, сессия истекла, и ZooKeeper сам удалил узел.

```
docker run --rm -it --name zkcli --network lab zookeeper:3.9.6 zkCli.sh -server zoo1:2181
create -e /demo hello
ls /
```



Узел /demo создан в окне клиента

```
[demo, zookeeper]
[zookeeper]
```



Клиент завершён, через 35 секунд узла нет

2. Координатор

Координатор `coordinator.py` считает пары навыков варианта 9 так же, как в ДЗ 3: читает CSV из RustFS кусками по 8 МБ, отбирает вакансии варианта и считает пары навыков. К этому добавлено:

- **Выбор лидера.** Обе копии встают в очередь на эфемерных последовательных узлах в `/hw4/RUN/election`. Считает только лидер, вторая ждёт. Таймаут сессии 10 секунд.
- **Эпоха.** Став лидером, координатор записывает узел `/hw4/RUN/epoch`, его новая версия и есть эпоха лидера: у первого 1, у следующего 2.
- **Чекпоинт.** После каждого куска в RustFS сохраняется смещение в файле и накопленные счётчики. Новый лидер продолжает с этого места.
- **Публикация.** Результат пишется в RustFS, а в узел `/hw4/RUN/result` пишется указатель на него. Без защиты указатель просто перезаписывается. С защитой `FENCING=1` запись идёт одной транзакцией: проверка, что версия узла эпохи всё ещё равна моей эпохе, и запись указателя.

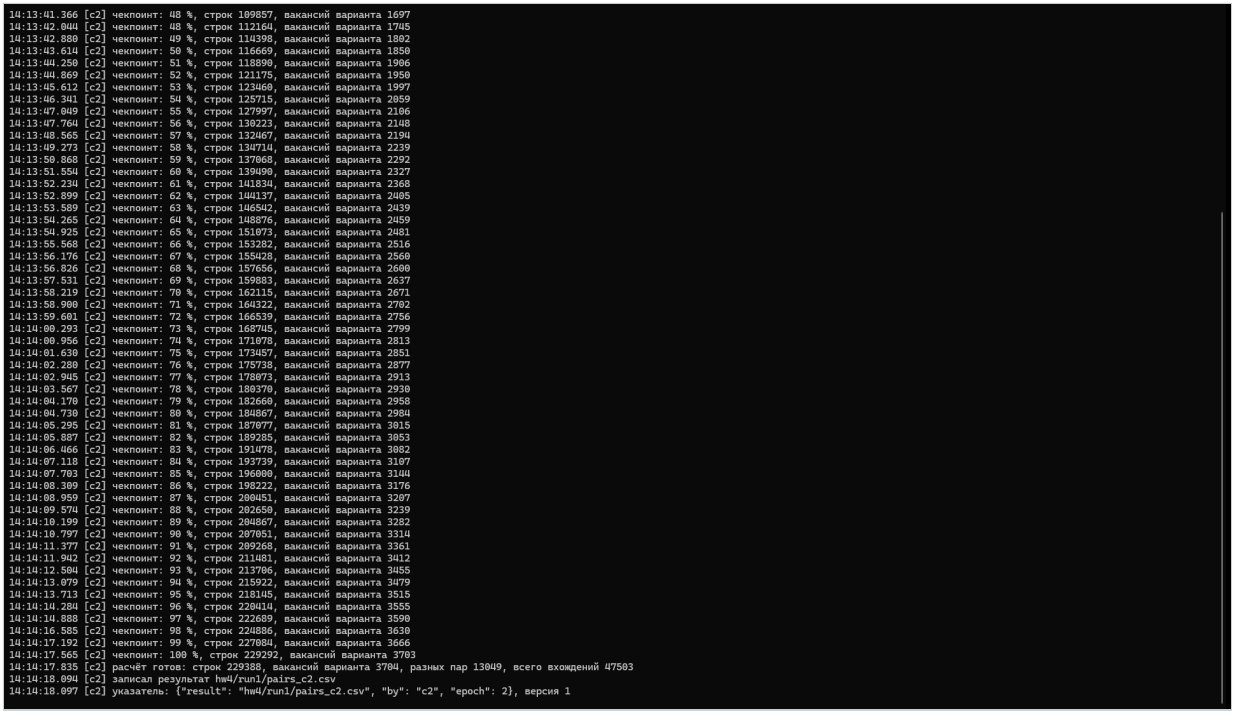
Лидер c1 приостанавливался командой `docker pause coord1` примерно на 30 % расчёта и возобновлялся командой `docker unpause coord1` после того, как запасной c2 записал результат.

3. Приостановка лидера без защиты

```
$env:RUN = "run1"; $env:FENCING = "0"
docker compose --profile coord up -d coord1
docker compose --profile coord up -d coord2
docker pause coord1          # пауза лидера (UTC): 14:13:23.328
docker unpause coord1
```

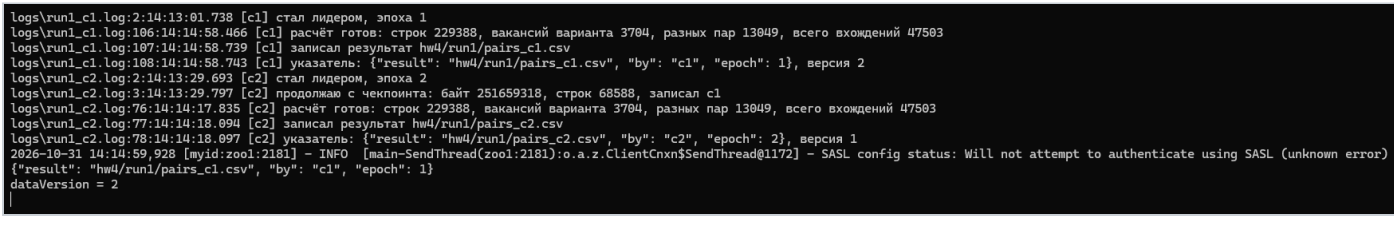
```
14:13:01.693 [c1] запущен, FENCING=0, жду лидерства
14:13:01.738 [c1] стал лидером, эпоха 1
14:13:01.882 [c1] чекпоинта нет, начинаю с начала
14:14:58.466 [c1] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049, всего вхождений
47503
14:14:58.739 [c1] записал результат hw4/run1/pairs_c1.csv
14:14:58.743 [c1] указатель: {"result": "hw4/run1/pairs_c1.csv", "by": "c1", "epoch": 1}, версия 2

14:13:12.503 [c2] запущен, FENCING=0, жду лидерства
14:13:29.693 [c2] стал лидером, эпоха 2
14:13:29.797 [c2] продолжаю с чекпоинта: байт 251659318, строк 68588, записал c1
14:14:17.835 [c2] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049, всего вхождений
47503
14:14:18.094 [c2] записал результат hw4/run1/pairs_c2.csv
14:14:18.097 [c2] указатель: {"result": "hw4/run1/pairs_c2.csv", "by": "c2", "epoch": 2}, версия 1
```



Журнал c2: продолжил с чекпоинта и записал результат

```
logs\run1_c1.log:2:14:13:01.738 [c1] стал лидером, эпоха 1
logs\run1_c1.log:106:14:14:58.466 [c1] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049,
всего вхождений 47503
logs\run1_c1.log:107:14:14:58.739 [c1] записал результат hw4/run1/pairs_c1.csv
logs\run1_c1.log:108:14:14:58.743 [c1] указатель: {"result": "hw4/run1/pairs_c1.csv", "by": "c1", "epoch":
1}, версия 2
logs\run1_c2.log:2:14:13:29.693 [c2] стал лидером, эпоха 2
logs\run1_c2.log:3:14:13:29.797 [c2] продолжаю с чекпоинта: байт 251659318, строк 68588, записал c1
logs\run1_c2.log:76:14:14:17.835 [c2] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049,
всего вхождений 47503
logs\run1_c2.log:77:14:14:18.094 [c2] записал результат hw4/run1/pairs_c2.csv
logs\run1_c2.log:78:14:14:18.097 [c2] указатель: {"result": "hw4/run1/pairs_c2.csv", "by": "c2", "epoch":
2}, версия 1
{"result": "hw4/run1/pairs_c1.csv", "by": "c1", "epoch": 1}
dataVersion = 2
```



Две строки «записал результат», указатель от c1 с версией 2

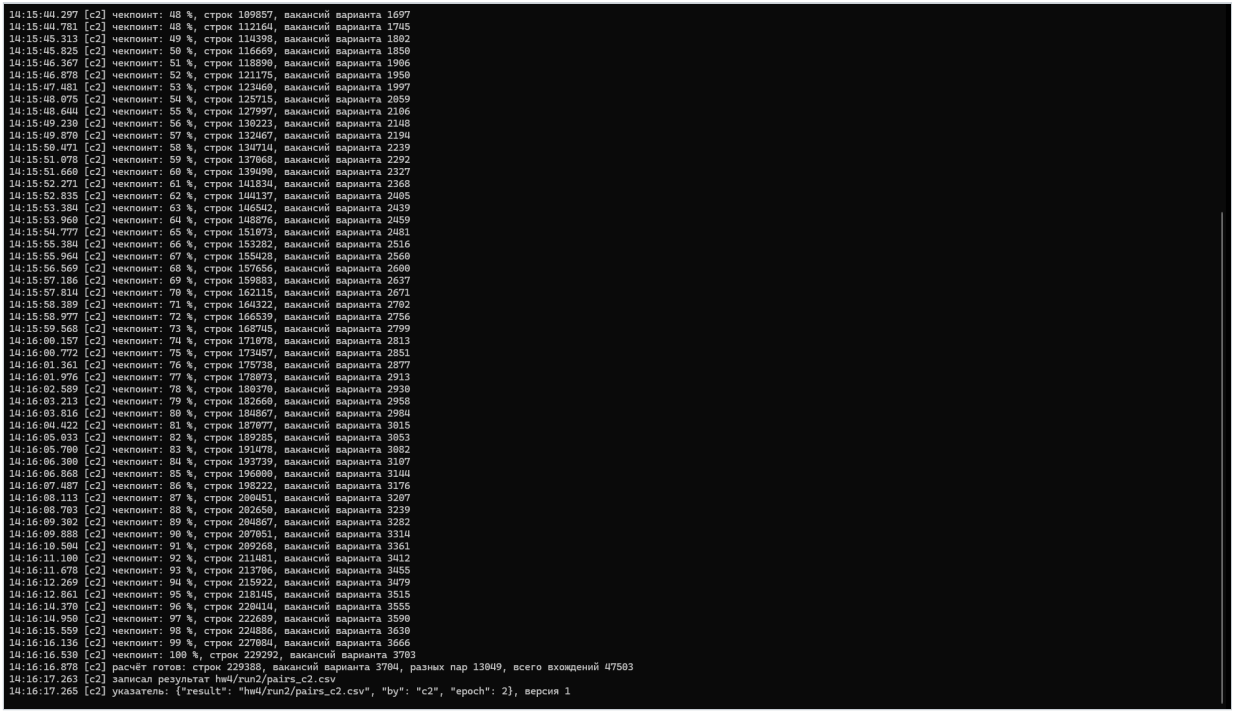
c2 стал лидером с эпохой 2 и продолжил с чекпоинта c1: с байта 251 659 318, после 68 588 строк. Оба координатора записали результат, последним c1, поэтому указатель теперь показывает на его файл, а версия узла 2. Устаревший лидер затёр результат действующего.

4. Приостановка лидера с защитой по эпохе

```
$env:RUN = "run2"; $env:FENCING = "1"
docker compose --profile coord up -d coord1
docker compose --profile coord up -d coord2
docker pause coord1          # пауза лидера (UTC): 14:15:28.061
docker unpause coord1
```

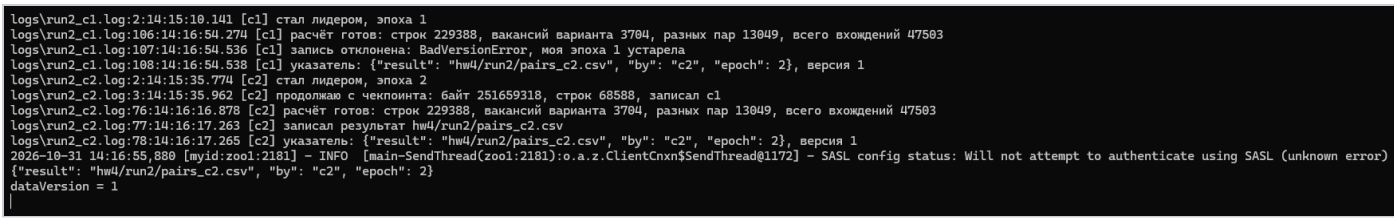
```
14:15:10.112 [c1] запущен, FENCING=1, жду лидерства
14:15:10.141 [c1] стал лидером, эпоха 1
14:15:10.247 [c1] чекпоинта нет, начинаю с начала
14:16:54.274 [c1] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049, всего вхождений
47503
14:16:54.536 [c1] запись отклонена: BadVersionError, моя эпоха 1 устарела
14:16:54.538 [c1] указатель: {"result": "hw4/run2/pairs_c2.csv", "by": "c2", "epoch": 2}, версия 1

14:15:19.073 [c2] запущен, FENCING=1, жду лидерства
14:15:35.774 [c2] стал лидером, эпоха 2
14:15:35.962 [c2] продолжаю с чекпоинта: байт 251659318, строк 68588, записал c1
14:16:16.878 [c2] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049, всего вхождений
47503
14:16:17.263 [c2] записал результат hw4/run2/pairs_c2.csv
14:16:17.265 [c2] указатель: {"result": "hw4/run2/pairs_c2.csv", "by": "c2", "epoch": 2}, версия 1
```



Журнал c2 во втором прогоне

```
logs\run2_c1.log:2:14:15:10.141 [c1] стал лидером, эпоха 1
logs\run2_c1.log:106:14:16:54.274 [c1] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049,
всего вхождений 47503
logs\run2_c1.log:107:14:16:54.536 [c1] запись отклонена: BadVersionError, моя эпоха 1 устарела
logs\run2_c1.log:108:14:16:54.538 [c1] указатель: {"result": "hw4/run2/pairs_c2.csv", "by": "c2", "epoch":
2}, версия 1
logs\run2_c2.log:2:14:15:35.774 [c2] стал лидером, эпоха 2
logs\run2_c2.log:3:14:15:35.962 [c2] продолжаю с чекпоинта: байт 251659318, строк 68588, записал c1
logs\run2_c2.log:76:14:16:16.878 [c2] расчёт готов: строк 229388, вакансий варианта 3704, разных пар 13049,
всего вхождений 47503
logs\run2_c2.log:77:14:16:17.263 [c2] записал результат hw4/run2/pairs_c2.csv
logs\run2_c2.log:78:14:16:17.265 [c2] указатель: {"result": "hw4/run2/pairs_c2.csv", "by": "c2", "epoch":
2}, версия 1
{"result": "hw4/run2/pairs_c2.csv", "by": "c2", "epoch": 2}
dataVersion = 1
```



Запись c1 отклонена с BadVersionError, указатель от c2 с версией 1

Когда c1 очнулся и попробовал опубликовать результат, версия узла эпохи была уже 2, а не 1. Транзакция не прошла с BadVersionError, указатель остался от c2, версия 1. Результат один.

5. Результаты и время смены лидера

	строк CSV	вакансий варианта	разных пар	всего вхождений
ДЗ 3	229 388	3 704	13 049	47 503
прогон 1, c1 и c2	229 388	3 704	13 049	47 503
прогон 2, c1 и c2	229 388	3 704	13 049	47 503

Результаты обоих координаторов в обоих прогонах совпали с ДЗ 3, хотя c2 начинал не с начала, а с чекпоинта.

	пауза по часам Windows	последняя запись c1 до паузы	c2 стал лидером	от паузы	от последней записи c1
прогон 1	14:13:23.328	14:13:20.022	14:13:29.693	6,4 с	9,7 с
прогон 2	14:15:28.061	14:15:25.560	14:15:35.774	7,7 с	10,2 с

Время паузы записано по часам Windows, а журналы координаторов пишутся по часам виртуальной машины Docker. Разница 6,4 и 7,7 секунды меньше таймаута сессии 10 секунд, чего быть не может: значит, часы расходятся на несколько секунд. По часам самих контейнеров от последней записи c1 до лидерства c2 прошло 9,7 и 10,2 секунды. Это таймаут сессии 10 секунд: ZooKeeper ждёт его, удаляет эфемерный узел c1, и c2, который ждал на следующем узле очереди, сразу становится лидером.

6. Ответы на вопросы

Почему серверов три, а не два и не четыре

ZooKeeper работает, пока живо большинство серверов. Из двух серверов большинство это оба, поэтому отказ любого останавливает сервис, и два сервера надёжнее одного только в том, что данные лежат в двух местах. Три сервера переживают отказ одного: остаются два из трёх. Четыре сервера тоже переживают отказ только одного, потому что двое из четырёх это не большинство, а каждая запись должна дождаться подтверждения от трёх серверов вместо двух. То есть четвёртый сервер добавляет нагрузку и не добавляет устойчивости. Это видно и в этапе 2: без одного сервера ансамбль работал, без двух нет.

Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

ZooKeeper не может остановить зомби. Пока c1 стоял на паузе, ZooKeeper удалил его узел и отдал лидерство c2, но сам процесс c1 об этом не знал: после паузы он продолжил считать с того же места и пошёл записывать результат. Между моментом, когда c1 последний раз убедился, что он лидер, и моментом записи может пройти сколько угодно времени, поэтому проверка на стороне писателя ничего не гарантирует. Гарантию даёт только тот, кто принимает запись: в момент самой записи он сравнивает эпоху писателя с текущей и отклоняет старую. В прогоне 2 получателем был узел указателя, и проверка шла в той же транзакции, что и запись. Если бы результат писался в базу или хранилище, проверять эпоху пришлось бы им.